

Introducere în programare paralelă și distribuită - standardul MPI.

Obiectivele lucrării

Lucrarea prezintă conceptul de calcul paralel și distribuit, o clasificare generală a sistemelor paralele de calcul, un set de noțiuni introductive ale standardului *MPI* (*Message Passing Interface*), urmărind familiarizarea cu conceptele specifice programării bazată pe biblioteci de mesaje. Sarcinile laboratorului includ:

- ⇒ Exercițiul 1 – Conceptul de calcul paralel și distribuit.
- ⇒ Exercițiul 2 – Clasificarea generală a sistemelor paralele de calcul.
- ⇒ Exercițiul 3 – Software pentru transfer de mesaje.
- ⇒ Exercițiul 4 – Standardul *MPI* (funcții de management mediu, comunicatori și grupuri de procese, funcții de comunicație punct la punct, instalarea și utilizarea bibliotecii *MPI* în .NET, aplicații în *MPI*).

Exercițiul 1 – Conceptul de calcul paralel și distribuit

Calculul paralel și distribuit reprezintă o *strategie* pentru a rezolva cât mai *rapid* probleme de o *complexitate ridicată* (volum mare de date și de calcule necesare). O anumită aplicație poate fi executată serial într-o succesiune de pași, sau poate fi descompusă în mai multe sub-probleme care se execută **în paralel** pe mai multe procesoare. Ideea de bază este aceea că problemele de rezolvat pot fi uneori împărțite în mai multe probleme mai simple, de natură similară sau identică între ele, care pot fi rezolvate simultan. În general, conceptul de *parallelism* presupune:

- Împărțirea (descompunerea) algoritmului sau a datelor în mai multe părți de dimensiuni mai reduse;
- alocarea părților la mai multe procesoare ce vor lucra în paralel;
- coordonarea activității și comunicării între procesoare;
- evitarea descompunerii algoritmului sau datelor inițiale în părți de dimensiuni prea mici, pentru ca nu cumva timpul în care li se comunică procesoarelor ce au de făcut să devină mai mare decât timpul pe care acestea îl consumă lucrând efectiv.

Probleme tipice în care se utilizează calcul paralel:

- proiectarea asistată de calculator: construcții de automobile, de avioane;
- proiectarea automată a circuitelor integrate – probleme de optimizare;
- modelarea și simularea diferitelor fenomene (planificarea economică, fizica nucleară, chimie moleculară, meteorologie, astronomie, seismologie);
- prelucrarea imaginilor;
- recunoașterea vorbirii.

Exercițiul 2 – Clasificarea generală a sistemelor paralele de calcul

De-a lungul evoluției calculatoarelor s-au făcut mai multe clasificări ale acestora. Cele mai cunoscute dintre acestea sunt: clasificarea lui *Flynn*; clasificarea lui *Hockney*; clasificarea lui *Schwartz*, clasificarea lui *Händler* și clasificarea *Flynn-Johnson*. În acest laborator vom prezenta clasificarea calculatoarelor paralele realizată de *Flynn*.

În anul 1966, *Flynn* a făcut o clasificare a arhitecturilor de calculatoare după natura *fluxului de instrucțiuni* (FI) și a *fluxului de date* (FD). Clasificarea cuprinde 4 tipuri de calculatoare ce vor fi prezentate în continuare.

1. Single Instruction Stream, Single Data Stream – **SISD**.
2. Single Instruction Stream, Multiple Data Stream – **SIMD**.
3. Multiple Instruction Stream, Single Data Stream – **MISD**.
4. Multiple Instruction Stream, Multiple Data Stream – **MIMD**.

Exercițiul 3 – Software pentru transfer de mesaje

În acest laborator discutăm numai despre programarea calculatoarelor **MIMD** cu memorie distribuită. Deoarece fiecare procesor are memoria sa proprie, singura modalitate de comunicare între procesoare este *transmiterea de mesaje*. Operația de bază este cea în care un procesor sursă P_s transmite un mesaj M conținând date din memoria sa M_s unui procesor destinație P_d , care stochează datele în memoria sa M_d . În general, vom presupune că procesoarele sursă și destinație sunt vecine, adică există o legătură fizică între ele, pe care circulă mesajul.

Abordarea bazată pe transferul de mesaje permite exploatarea *paralelismului distribuit*, folosind programarea paralelă. Transferul mesajelor între sisteme necesită cooperarea între emițător și receptor, având

la bază o serie de modalități specifice de *descriere a datelor, de identificare a proceselor și a mesajelor*. În sistemele cu transmitere de mesaje utilizarea termenului de comunicare referă mai multe operații de interacțiune și anume *comunicarea, sincronizarea și agregarea*. Modurile de interacțiune sunt direct corelate cu modurile de comunicare, iar comunicația apare de obicei între procese ce aparțin aceluiași grup, există însă biblioteci software pentru care este permisă și comunicarea între grupuri diferite de procese.

De-a lungul timpului au fost dezvoltate o serie de biblioteci software pentru transferul de mesaje. Cele mai importante, cât și caracteristicile lor de bază, sunt prezentate în tabelul 1.1.

Tabelul 1.1 *Sisteme software cu transfer de mesaje*

Nume	Caracteristici
Linda	Este un limbaj realizat la Universitatea Yale de către participanții la proiectul Yale-Linda Group. Proiectul avea ca scop obținerea unui limbaj de programare paralel de mare performanță și interes general.
Fortran-M	Este un limbaj realizat de „Argonne National Lab” caracterizat prin modularitate și determinare.
ORCA	Limbaj de programare paralelă destinat sistemelor distribuite și are la bază modelul partajat data-obiect. A fost realizat la Vrije Universiteit Amsterdam și funcționează pe un număr mare de platforme: Amoeba 5.0, SunOS 4.1, Solaris 2, Linux, BSDI BSD/OS, CM-5, SP-2, Parix (T800 and PowerXplorer), Myrinet.
NESL	Limbaj dezvoltat de către Universitatea Carnegie-Mellon în cadrul proiectului SCANDAL. Principalul scop al limbajului NESL a fost acela de a face aplicațiile paralele portabile.
Parsec	Este un limbaj de simulare bazat pe C dezvoltat la Universitatea din California. A fost realizat pentru simularea unor modele matematice complexe atât în programarea secvențială cât și în cea paralelă. Poate fi folosit și ca limbaj de programare paralel.
ZPL	Reprezintă un limbaj paralel construit în jurul ideilor de performanță și portabilitate.
OpenHPC++	Reprezintă un set de instrumente software și biblioteci care extind funcționalitatea limbajului C++ astfel încât acesta să poată fi folosit și pentru aplicații paralele. HPC++ este construit pe liniile trasate de tehnologia CORBA permițând interconectarea unui număr mare de resurse de calcul eterogene.
Parallax III	Limbaj de programare structurat pentru mașini de tip SIMD dezvoltat de Thomas Braunl în 1989. Este construit pe baza limbajului secvențial Modula 2. Programele scrise în Parallax-III sunt portabile deoarece sunt realizate pentru o configurație virtuală de procesoare interconectate și pot fi rulate indiferent de hardware-ul existent.
Nx	Are la bază Intel hypercube MPP (realizat de Intel).
P4	Integrează memoria partajată și transfer de mesaje (Argonne

	National Lab).
PVM	Parallel Virtual Machine (PVM) reprezintă o bibliotecă de funcții scrise în limbajul C sau în FORTRAN cu ajutorul cărora se pot realiza aplicații paralele în rețele de calculatoare. A fost realizat de Oak Ridge National Lab .
MPI	Message Passing Interface (MPI) reprezintă o bibliotecă de funcții (un API) similar cu PVM.
Unify	Integrează ambele tipuri de apeluri: PVM–MPI.

Limbajele enumerate mai sus sunt câteva dintre multele limbaje folosite la ora actuală în domeniul procesării paralele. Motivul pentru care se folosesc atât de mult limbaje este faptul că sunt dependente de arhitectura și de obținerea de performanțe ridicate.

Exercițiul 4 – Standardul MPI (funcții de management mediu, comunicatori și grupuri de procese, funcții de comunicație punct la punct, instalarea și utilizarea bibliotecii MPI în .NET, aplicații în MPI)

MPI (Message Passing Interface) este un standard ce descrie primitive de comunicație – în contextul unui model de programare SPMD (Single Program Multiple Data) cu comunicație prin mesaje – apărut la mijlocul anilor '90. Scrierea unui program folosind rutinele de comunicație MPI asigură *portabilitatea*, eficiența comunicației fiind asigurată de rutinele MPI.

MPI este standardul pentru transmiterea de mesaje în sistemele paralele. Standardul MPI este descris pentru limbajele FORTRAN și C. În acest laborator vom folosi numai convențiile de apelare a funcțiilor C. Succesul MPI se datorează înglobării caracteristicilor existente în bibliotecile de schimb de mesaje existente până la momentul elaborării standardului (PVM, P4, Zipcode, PARMACS, TCGMSG, Express). MPI reprezintă o *bibliotecă standardizată portabilă*, implementată într-un mod independent de limbaj, ce oferă un set de funcții de legătură cu limbajele C și Fortran. Specificația definită de standard nu conține caracteristicile unui anumit sistem de operare sau mașină hardware fapt ce îi conferă *portabilitate*.

Standardul MPI definește o interfață de programare prin schimb explicit de mesaje, interfață ce poate fi utilizată pentru scrierea de aplicații pentru sisteme de tip MIMD cu memorie distribuită, respectiv mașini MIMD cu memorie partajată. Este important de înțeles faptul că **MPI este o bibliotecă de funcții, nu un limbaj**, astfel MPI definește nume de funcții, secvențe de apel și rezultate pentru un set de funcții care pot fi apelate în programe precum C, C++ și Fortran. Compilarea programelor care folosesc această bibliotecă MPI poate fi realizată cu un compilator standard C,

Fortran sau cu alte compilatoare performante cum ar fi OpenMP), iar în faza de editare a legăturilor este introdusă și biblioteca MPI.

Principalele caracteristici care au determinat o largă răspândire a standardului MPI sunt:

- ⇒ *oferă un set de funcții de referință pentru proiectanții de hardware;*
- ⇒ *conține un set de rutine pentru lucrul cu grupuri de procese fapt ce permite scheme de adresare specifice aplicațiilor;*
- ⇒ *permite comunicație punct-la-punct în medii eterogene (rețele diverse);*
- ⇒ *permite comunicație colectivă și operații de calcul globale în cadrul unui grup de procese;*
- ⇒ *oferă contexte de comunicare și comunicatori pentru dezvoltarea de biblioteci de funcții utilizabile în programe paralele;*
- ⇒ *conține rutine pentru definirea de topologii virtuale de grup;*
- ⇒ *permite implementarea de mecanisme specifice pentru depanarea și profilarea programelor;*
- ⇒ *oferă portabilitate și ușurință în folosire, semantica sa permite implementări diverse;*
- ⇒ *este performant - permite gestionarea ierarhiei de memorie, iar operațiile colective induc scalabilitatea sistemului;*
- ⇒ *oferă simplitate și completitudine deși posedă un număr extrem de mare de funcții. Astfel, aplicații relativ complexe pot fi programate bazat pe un set minimal de 6 funcții esențiale. Orice algoritm paralel poate fi implementat folosind biblioteca de funcții.*
- ⇒ *este modular, folosirea comunicatorilor reflectă facilitățile de implementare bazate pe componente.*
- ⇒ *este compozabil fie în sensul formării unor obiecte cu domenii suport mai mari, fie prin asociere (suprapunere) de proprietăți, pentru formarea obiectelor multidimensionale, interacționează cu compilatoare, depanatoare.*

După publicarea standardului (1994), MPI a fost implementat pe multe sisteme paralele începând cu supercomputere masiv paralele (MPP), servere cu multiprocesare simetrică (SMP), clustere de stații de lucru sau PC-uri. Noul standard (MPI – 2) a completat funcționalitatea primei versiuni 1.1 cu o serie de primitive dintre care cele mai importante sunt: *funcții de creare dinamică de procese*, *funcții pentru depanare*, *funcții pentru gestiunea proceselor*, *respectiv operații de intrare/ieșire paralele*. Inițial, standardul nu impunea interoperabilitatea versiunilor de MPI, motiv pentru

care a fost practic imposibilă cooperarea între programe scrise folosind implementări diferite de MPI, însă implementările recente ale standardului oferă suport complementar pentru interoperabilitate.

O serie de implementări din domeniul public sau comercial au apărut ca urmare a procesului de standardizare, cea mai reprezentativă și performantă (deoarece rulează în mod nativ, fără a utiliza socket-urile TCP-IP) fiind MSMPI (Microsoft Message Passing Inteface) o implementare portabilă pe diverse mașini (MPP, SMP, cluster). Setul de concepte ortogonale ce conferă funcționalitatea sa deosebit de puternică include: definirea de tipuri de date specifice diferitelor mesaje, abstracțiunile *comunicator* și *topologii virtuale*, operații de comunicare cu semantica variată.

Funcții de management mediu MPI

Stilul de programare care poate fi folosit cu MPI se bazează fie pe modelul **SPMD**, caz în care există un singur program, astfel procesele MPI provin din același program, dar execută porțiuni diferite de cod selectate pe baza unor instrucțiuni de condiție, fie pe modelul **MIMD** ce presupune existența de programe diferite pentru fiecare procesor (figura 1.9).

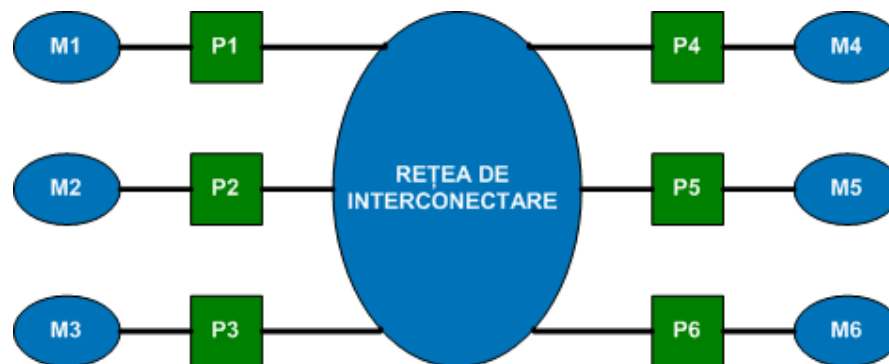


Figura 1.1 Modelul MPI

Standardul MPI a fost proiectat astfel încât programatorul începător să nu simtă complexitatea de la primele programe. Debutul în lumea MPI se poate face cunoscând doar un set minimal de funcții ce reprezintă nucleul suficient pentru scrierea unei aplicații (figura 1.10). Încadrându-se în modelul SPMD, un program MPI are forma obișnuită a unui program secvențial. Apelul funcțiilor de bibliotecă necesită includerea antetului *mpi.h* ce conține constantele referite în rutinele MPI (tipuri de date, comunicatori, erori, ...) și interfețele funcțiilor MPI.

MPI_Init (int argc, char*argv[]) este funcția care inițializează variabilele locale MPI și înștiințează sistemul global despre execuția unui nou proces MPI. Funcția acționează și ca barieră de inițializare, în sensul că nici un proces MPI nu își poate continua execuția până când toate celelalte procese nu au executat partea lor de inițializare. Această funcție trebuie apelată în orice program MPI înainte de orice altă funcție MPI și trebuie să fie chemată o singură dată într-un program. Argumentele *argc/argv* sunt citite la nivel de linie de comandă pentru programul C și pot avea importanță sau nu în program în funcție de modul de realizare a acestuia.

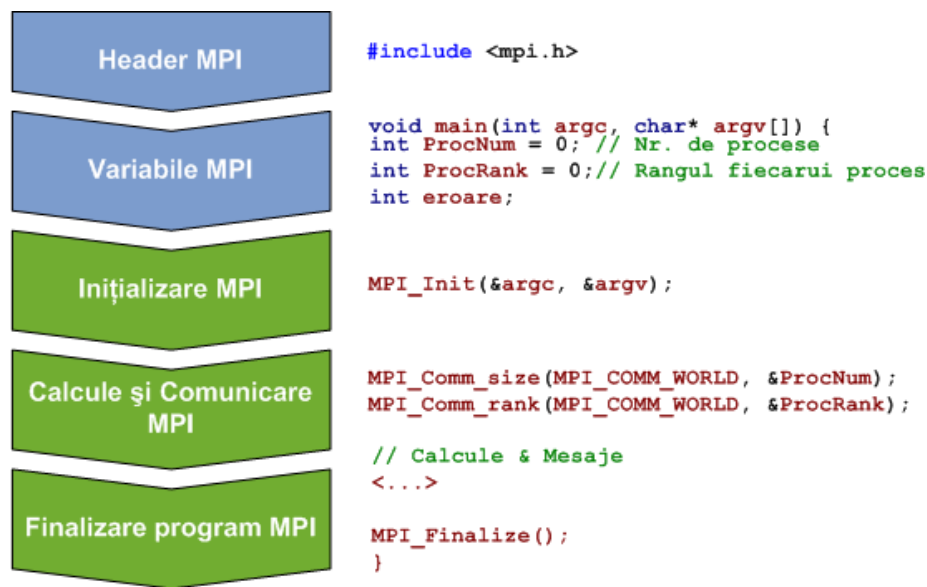


Figura 1.2 Structura unui program MPI

MPI_Comm_rank(MPI_COMM_WORLD,&ProcRank) returnează identificatorul (rangul) procesului MPI în cadrul grupului specificat în lista de parametri (în cazul prezentat în figura 1.10, comunicatorul asociat se numește *MPI_COMM_WORLD*). O rutină MPI se execută în cadrul unui grup de procese, numit *comunicator*. Inițial, există un singur comunicator, numit *MPI_COMM_WORLD*, conținând toate procesele pe care se execută programul. În cadrul unui comunicator, procesele sunt numerotate de la 0 la $p-1$, unde p reprezintă numărul de procese din comunicator. Un proces poate afla adresa proprie din comunicator (numită *rang* în MPI), apelând rutina *MPI_Comm_rank(...)*. Acest rang identifică în mod unic un proces și poate fi folosit pentru specificarea sursei sau destinației unui mesaj. Adresa procesului curent se salvează în variabila *ProcRank*.

`MPI_Comm_size(MPI_COMM_WORLD, &ProcNum)` returnează în variabila **`ProcNum`** numărul de procese care fac parte din grupul specificat ca parametru (grupul asociat comunicatorului **`MPI_COMM_WORLD`**). Un proces poate afla numărul de procese (numărul de procese se salvează în variabila **`ProcNum`**) din comunicator apelând rutina **`MPI_Comm_size(...)`**.

`MPI_Finalize()` anunță sistemul despre terminarea procesului MPI cu succes. Această rutină trebuie apelată de fiecare proces MPI.

Rutinele **`MPI_Init(...)`** și **`MPI_Finalize()`** trebuie apelate înaintea și respectiv, după orice alte rutine MPI. Aceasta este singura constrângere, în rest programatorul este liber să folosească rutinele MPI potrivite algoritmului pe care îl implementează.

Comunicatori și grupuri de procese

Comunicatorul integrează informațiile de context și informațiile de identificare a grupului de procese (figura 1.11). Toate comunicațiile MPI necesită un comunicator ca argument, iar procesele MPI nu pot comunica decât dacă *partajează* un comunicator. Fiecare comunicator identifică un grup reprezentând o listă de procese, fiecare proces fiind numerotat. Folosind **`MPI_COMM_WORLD`**, fiecare proces poate comunica cu oricare altul, iar grupul **`MPI_COMM_WORLD`** reprezintă mulțimea tuturor proceselor MPI. Structura internă a unui comunicator nu este accesibilă direct (este un obiect opac, fiind alocat de rutinele MPI și manipulat prin apelurile de bibliotecă).

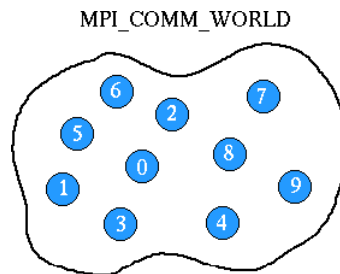


Figura 1.3 Comunicatorul implicit **`MPI_COMM_WORLD`**

Comunicatorul include grupul de procese, contextul și topologia proceselor alături de atributele asociate comunicatorului de către utilizator. Contextul nu poate fi accesat sau manipulat de utilizator dar este atașat mesajului, astfel mesajele punct la punct posedă contexte diferite de mesajele colective, cu scopul de a nu se interbloca.

Procesul și grupurile de procese. Procesul MPI este unitatea fundamentală de calcul, fiind constituit dintr-un *fir de control independent* și

un *spațiu de adrese privat*. Astfel un proces nu poate accesa direct variabilele definite în spațiul de adrese al altui proces, comunicarea fiind bazată pe transfer de mesaje. Contextul este partajat de un grup de procese. Procesele MPI se execută fără a avea însă mecanisme de încărcare a codului pe procesoare și atribuire a proceselor la procesoare. Aceste mecanisme sunt de obicei, în cadrul implementărilor curente, *preluate de la sistemul de operare pe care rulează* procesul MPI, fapt ce poate crea dificultăți de portare a acestor biblioteci între sisteme de operare diferite. MPI este proiectat a fi thread-safe, iar grupurile de procese definite în MPI sunt dinamice, apartenența la grup este statică însă suportă atașarea-detașarea din grup. Grupurile se pot suprapune fapt ce permite unui proces să fi membru al mai multor grupuri simultan (figura 1.12).

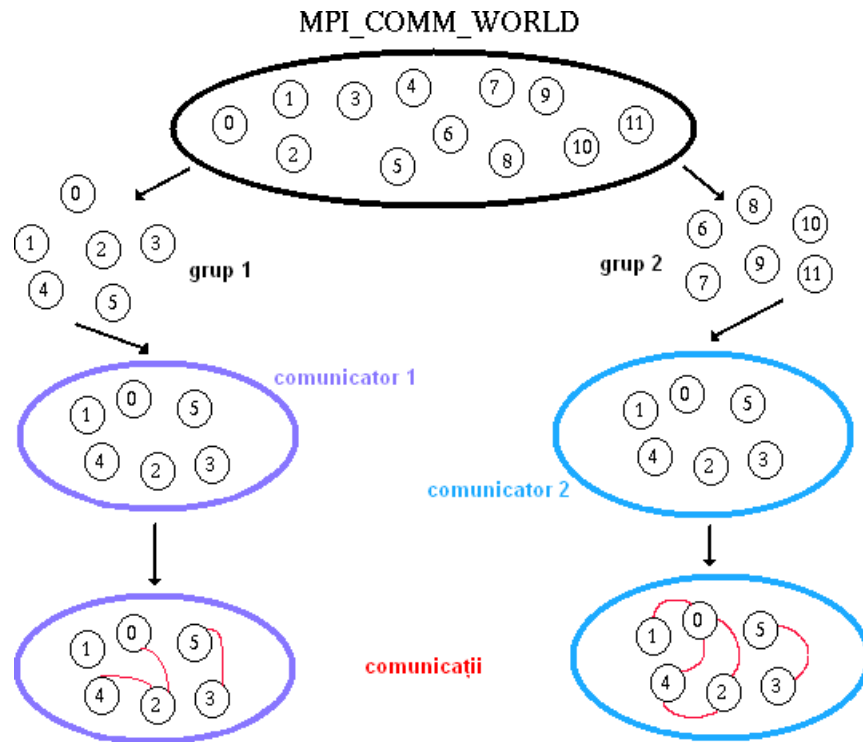


Figura 1.4 Comunicatori și grupuri de procese

Procesele sunt identificate în cadrul grupului printr-o variabilă de tip întreg (*ProcRank*), care ia valori între 0 și $p-1$, unde p este numărul proceselor din grup.

Funcții de comunicație punct la punct

Scopul principal al bibliotecii MPI este acela de a oferi un set de funcții pentru transmisia de mesaje între diferite procese MPI, motiv pentru care înglobează un set de mecanisme de comunicație preluate din alte sisteme (PVM, Linda), pe care însă le completează cu propriile abstracțiuni. Specificația oferă un set de rutine pentru transferul cooperativ de mesaje punct la punct sau colectiv, rutine pentru sincronizarea proceselor, tipuri de date derivate pentru patternuri de acces la date necontigue, abilitatea de a crea subseturi de procesoare și topologii variate de procese.

Pentru a putea construi o aplicație paralelă performantă este necesar un control performant asupra transmiterii mesajelor. În acest sens, MPI definește patru moduri de comunicare.

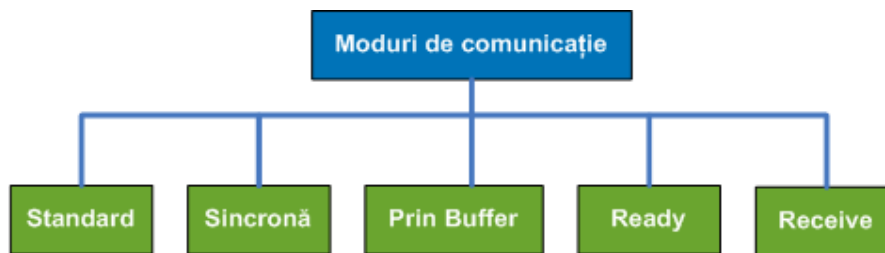


Figura 1.5 Moduri de comunicație în MPI

Comunicație **standard** - operația de trimitere a mesajului poate începe chiar dacă operația corespunzătoare de recepție nu a început.

Comunicație **sincronă** - operația de trimitere a mesajului poate începe oricând, dar nu se poate termina decât atunci când mesajul a ajuns la receptor.

Comunicație **prin buffer** – utilizatorul poate furniza sistemului un buffer pentru a permite ca operația de trimitere să se termine întotdeauna înainte ca mesajul să fie recepționat la destinație.

Comunicație **ready** - operația de trimitere a mesajului poate începe doar dacă operația corespunzătoare de recepție a început.

Să presupunem că procesorul P_s transmite un mesaj procesorului P_d . Deoarece fiecare procesor are fluxul său de instrucțiuni, momentele în care P_s apelează primitiva **send**, iar P_d primitiva **recv**, sunt în general diferite. Transmiterea efectivă a mesajului se face doar după ce ambele procesoare au apelat primitivile respective.

Se pune întrebarea: ce face procesorul care a apelat primul primitiva sa de comunicație, din momentul apelului și până când celălalt procesor apelează primitiva pereche? Sunt două răspunsuri posibile:

1. Așteaptă (fără să facă nimic) – cazul *comunicației blocante* (funcțiile de transmisie sau recepție se termină înainte ca informația să fie trimisă/recepționată);
2. Poate executa alte operații – cazul *comunicației neblocante* (după terminarea funcției, bufferul de transmisie sau recepție poate fi refolosit).

O operație de *send blocantă* va returna controlul doar atunci când zona de date ce a fost trimisă poate fi reutilizată, fără să afecteze datele primite de destinatar. O operație de *send blocantă* poate fi:

- sincronă (va returna controlul doar atunci când datele au ajuns efectiv la destinatar).
- Asincronă (se utilizează o zonă de tampon din sistem pentru salvarea datelor ce urmează a fi trimise).

O operație *receive blocantă* va returna controlul doar după ce datele au fost primite și pot fi folosite în program.

În tabelul 1.2 sunt prezentate funcțiile MPI utilizate pentru cele 4 moduri de comunicație prezentate în figura 1.13.

Tabelul 1.2 Numele funcțiilor MPI pentru cele 4 moduri de comunicație

Comunicație	Blocantă	Neblocantă
Standard	MPI_Send MPI_Recv	MPI_Isend MPI_Irecv
Sincronă	MPI_Ssend	MPI_Issend
Prin buffer	MPI_Bsend	MPI_Ibsend
Ready	MPI_Rsend	MPI_Irsend

Comunicația blocantă

Prezentăm acest tip de comunicație în figurile 1.14 și 1.15, presupunând că procesorul sursă este primul care apelează primitiva de comunicație (cazul în care destinatarul face primul apel, este obținut prin simetrie). În figura 1.14, la apelul primitivei **send** de către procesorul sursă, zona de memorie numită generic „mesaj” conține datele care trebuie transmise (ceea ce este marcat printr-un dreptunghi hașurat). Până în momentul apelării **recv** de către destinatar, procesorul sursă nu execută nici o operație, deci este blocat. După apelul **recv** are loc comunicația efectivă (marcată în figură cu o săgeată), după care zona „mesaj” devine liberă, adică poate fi utilizată în alte scopuri. Deoarece unul din procesoare așteaptă fără a face altceva, iar rutinele **send** și **recv** se termină doar după transferul mesajului, comunicația se numește *blocantă și sincronă*.

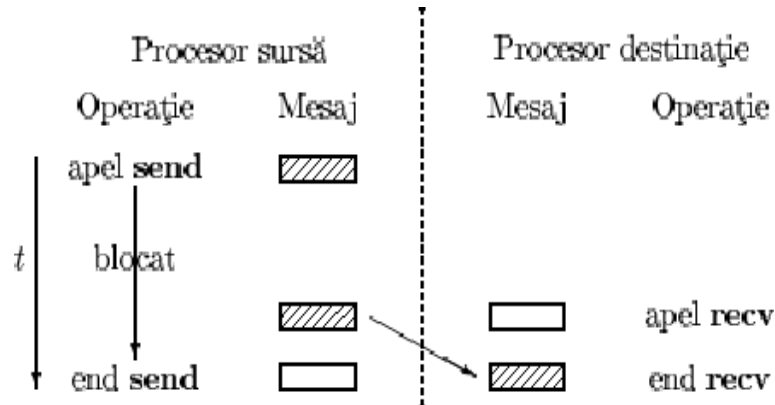


Figura 1.6 Comunicație blocantă între 2 procesoare – modul sincron

În figura 1.15, comunicația blocantă are loc printr-un buffer. Primitiva **send** mută mesajul din zona sa originală într-un buffer. După aceasta, execuția **send** se termină, zona de memorie a mesajului putând fi refolosită. Mesajul trece din buffer la destinație atunci când se execută **recv**. Procesorul sursă este blocat doar în timpul în care mesajul este copiat în buffer. Se observă că terminarea rutinei **send** la sursă nu este condiționată de apelul **recv** la destinație. Atât în varianta sincronă cât și în cea prin buffer, la terminarea execuției primitivei **send**, zona de memorie în care era memorat mesajul poate fi refolosită, iar la terminarea execuției **recv**, mesajul este recepționat în zona de memorie alocată acestui scop.

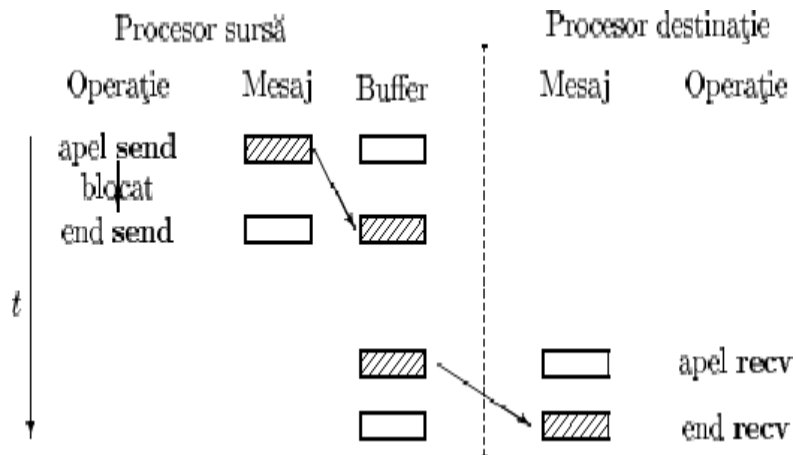


Figura 1.7 Comunicație blocantă prin buffer

O operație de comunicare este considerate *terminata local* în cadrul unui anumit proces, dacă acesta a terminat partea sa de calcul și comunicare.

O operație de comunicare este *terminată global*, dacă toate procesele implicate în comunicare au terminat local părțile ce le revin din aceasta.

Transmisie blocantă standard (MPI_Send). Forma *standard send* se încheie odată ce mesajul a fost trimis, mesaj care poate să ajungă sau nu la destinație, iar ignorarea acestui fapt poate duce la comportări inconsistente ale programului. Sintaxa:

```
MPI_Send (void *buffer, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm)
```

unde:

buffer este adresa datei ce va fi trimisă;

count este numărul de elemente de tipul *datatype* conținute de *buffer*;

dest reprezintă procesul destinație specificat prin intermediul rangului din cadrul comunicatorului *comm*;

tag este un marcator, la dispoziția programatorului, pentru a distinge între diferite tipuri de mesaje.

La execuția *MPI_Send* de către un procesor, mesajul aflat în memoria locală la adresa *buffer*, de lungime *count* și având tipul *datatype* (MPI are identificatori predefiniți pentru practic toate tipurile uzuale de date – aceștia vor fi prezentați pe larg în lucrarea 2) este transmis procesorului *dest*. Mesajului îi este asociată o etichetă (*tag*), care îl personalizează. Transmitia mesajului se face în cadrul comunicatorului *comm*, care trebuie să conțină și procesorul sursă și cel destinație.

Transmisie sincronă (MPI_Ssend – Synchronous send).

Synchronous send se folosește atunci când se dorește confirmarea primirii mesajului. Astfel, destinația trimite spre sursă o confirmare iar transmitia este considerată încheiată numai după primirea confirmării. Primitiva *send* se consideră încheiată atunci când o operație *receive* a fost lansată. În acest fel, bufferul aplicației este disponibil pentru a recepționa mesajul. Această formă de transfer se folosește atunci când este nevoie de siguranță a transmisiei, pentru a oferi comportament determinist. Sintaxa:

```
int MPI_Ssend (void * buffer, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm)
```

Transmisie prin buffer (MPI_Bsend – Buffered send).

Forma *Buffered send* folosește un buffer pentru trimiterea mesajului, dacă acesta nu poate fi trimis la momentul dorit. Avantajul față de metoda clasică îl constituie predictibilitatea recepției mesajului. Sintaxa:

```
int MPI_Bsend (void *buffer, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm)
```

Atașarea unui buffer se realizează cu ajutorul funcției:

```
int MPI_BUFFER_ATTACH (void *buffer, int size)
```

Numai un singur buffer poate fi atașat unui proces la un moment dat. Detașarea se realizează cu funcția:

```
int MPI_BUFFER_DETACH (void *buffer, int size).
```

După detașare, utilizatorul poate reutiliza sau dealoca spațiul ocupat de tampon. Unele operații, de atașare și detașare, au un argument de tip `void*`; acestea sunt folosite diferit – un pointer la tampon este transmis la `attach`; adresa pointerului este utilizată la `detach`, astfel că acest apel întoarce valoarea pointerului.

Transmisie ready (MPI_Rsend – Ready send). Formele *Ready send*, ca și *buffered send* se termină imediat, iar comunicarea este garantată a se efectua cu succes dacă o operație *receive* corespunzătoare a fost apelată. La expedierea mesajului, procesul expeditor pune mesajul în comunicator, sperând ca destinatarul să îl aștepte și să îl primească. Dacă acesta nu îl acceptă, mesajul poate fi abandonat sau se generează o eroare. Sintaxa:

```
int MPI_Rsend (void *buffer, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)
```

Recepție blocantă (MPI_Recv – Blocking receive). Sintaxa:

```
int MPI_Recv (void *buffer, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status status)
```

unde:

source este rangul expeditorului în grupul asociat comunicatorului *comm*; Ca și sursă, se poate specifica `MPI_ANY_SOURCE`, ceea ce este echivalent cu orice sursă;

tag specifică tipul mesajelor ce vor fi acceptate. Se poate, de asemenea, specifica `MPI_ANY_TAG` ca fiind identificator universal de mesaj.

Informația este returnată de către funcția *MPI_Recv* în variabila *status*. Acest argument poate fi testat direct pentru a afla sursa mesajului, dimensiunea și identificatorul de mesaj (de exemplu, poate fi necesară identificarea sursei unui mesaj recepționat folosind specificația `MPI_ANY_SOURCE`).

Comunicația neblokantă

Comunicația neblokantă începe transmisia unui mesaj cât mai repede și continuă cu alte calcule, iar apoi revine asupra confirmării la un moment de timp ulterior. Trebuie remarcat faptul că, dacă timpul ce se câștiga pentru calcule folosind acest mod de transmisie este mic, soluția nu este utilă, însă în situația contrară, performanța sistemului poate crește simțitor. Este permisă utilizarea combinată a funcțiilor de recepție emisie

neblocante/blocante, însă programatorul este responsabil de corectitudinea programului.

În modul neblokant, primitivele **send** și **recv** se termină imediat după apel, fără ca zona de memorie a mesajului să fie liberă la **send** sau să conțină mesajul la **recv**. Primitivele au doar rolul de a iniția comunicația, transferul efectiv fiind realizat mai târziu la un nivel inferior. După terminarea execuției primitivelor **send** sau **recv**, procesorul poate executa alte operații, în paralel sau concurrent cu comunicația. Și în modul neblokant, comunicația se poate desfășura prin buffer sau sincron, prima variantă fiind ilustrată în figura 1.16.

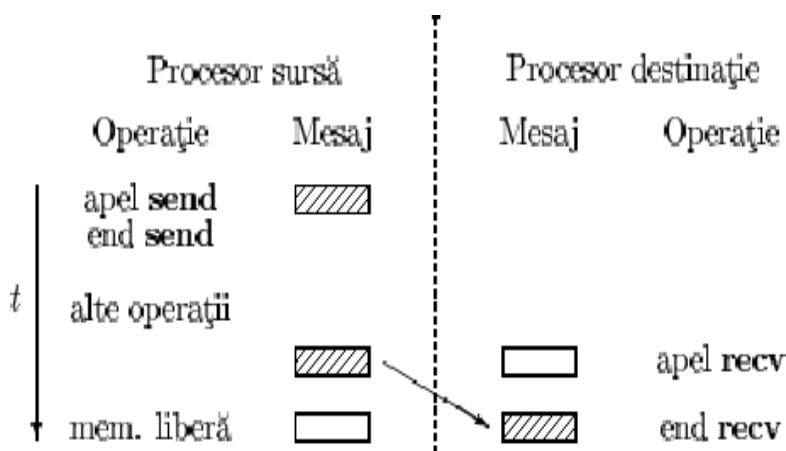


Figura 1.8 Comunicație neblokantă prin buffer

Este evident că primitivele de comunicație neblocante trebuie să fie însoțite de posibilitatea de a afla când se termină comunicația efectivă, mai exact de a afla când zona de memorie a mesajului poate fi refolosită de procesorul sursă sau poate fi utilizată de procesorul destinație. În acest scop se introduce primitiva **wait** de așteptare a terminării comunicației.

Așadar, utilizarea tipică a rutinelor neblocante constă în:

1. **send** (*date*, *dest*)
2. execută operații care nu modifică *date*
3. **așteaptă** terminarea **send**
4. modifică *date*

În mod evident, dacă **așteaptă** se execută imediat după **send** sau **recv**, efectul este același ca în cazul primitivelor blocante. Termenul de *comunicație neblokantă* semnifică întoarcerea funcției imediat după inițializarea transferului, nu neapărat terminarea lui, astfel buffer-ul folosit într-o transmisie neblokantă nu poate fi utilizat în siguranță după revenirea din funcție. Cele patru modalități de trimitere a mesajelor în forma blocantă

(*MPI_Send*, *MPI_Ssend*, *MPI_Bsend*, *MPI_Rsend*) au echivalente în forma neblocaanta (*MPI_Isend*, *MPI_Issend*, *MPI_Ibsend*, *MPI_Irsend*). Aceste funcții au un argument suplimentar, reprezentând un obiect de reîntoarcere care poate fi transmis unei funcții de verificare. Spre exemplu, funcția pentru trimiterea unui mesaj în forma sincronă neblocaantă este:

```
int MPI_Isend (void *buffer, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm, MPI_Request *req);
```

iar funcția pereche pentru recepționarea mesajului este:

```
int MPI_Irecv (void *buffer, int count, MPI_Data type  
datatype, int source, int tag, MPI_Comm comm, MPI_Request  
*req);
```

Semnificația argumentelor celor două funcții este aceeași ca și la perechile lor blocante, însă, suplimentar, cele două funcții posedă argumentul *req*. Funcțiile alocă un obiect request și returnează un pointer la acest obiect în variabila *req*. Acest obiect este utilizat ca argument pentru funcțiile *MPI_Test* sau *MPI_Wait* pentru a identifica operațiile a căror stare este interogată, respectiv pentru care se așteaptă terminarea (funcția de recepție neblocaantă nu posedă un argument de stare *status* similar funcțiilor blocante).

Testarea comunicației

Pentru modurile de comunicație prezentate mai sus, MPI definește funcții care execută testul de terminare a operației. Acest test se poate efectua asupra terminării tuturor operațiilor de comunicație dintr-o mulțime (*MPI_Waitall*), a unei operații dintr-o mulțime (*MPI_Waitany*) sau cel puțin a unei operații (*MPI_Waitsome*). Testele sunt de două tipuri:

- tip *WAIT* – aceste funcții se blochează până la încheierea comunicației, fiind folosite atunci când datele din mesaj sunt necesare pentru calcule, sau bufferul de comunicație trebuie reutilizat. Astfel un apel neblocaant urmat de un test de tip *WAIT* este echivalent cu o comunicație de tip blocantă. Sintaxa este: `int MPI_WAIT (MPI_Request *request, MPI_Status *status).`
- tip *TEST* – funcție ce returnează *true* sau *false*, dacă comunicația s-a încheiat sau nu fără a bloca sistemul. Aceste funcții sunt utile atunci când se dorește confirmarea comunicației. Sintaxa este: `int MPI_TEST (MPI_Request *request, int flag, MPI_Status *status).`

De multe ori, mai multe comunicații neblocaante sunt inițiate în același timp, iar biblioteca de funcții MPI oferă suport pentru testarea comunicațiilor multiple în același timp. Există trei tipuri de funcții care:

- testează pentru completitudine *toate* comunicațiile;
- testează pentru completitudine *oricare* comunicație;
- testează pentru completitudine *anumite* comunicații.

Fiecare din cele trei are ambele forme: WAIT și TEST (tabelul 1.3).

Tabelul 1.3 Funcții MPI pentru testarea comunicației

Test	Wait	Test
Cel puțin una, returnează exact una	MPI_Waitany	MPI_Testany
Toate	MPI_Waitall	MPI_Testall
Cel puțin una, returnează toate cele terminate	MPI_Waitsome	MPI_Testsome

int MPI_Waitall (int *count, MPI_Request array_of_requests[], MPI_Status array_of_statuses[]) - testează pentru completitudine toate comunicațiile specificate. Funcția se blochează până când toate comunicațiile specificate în *array_of_requests* s-au terminat. Starea comunicațiilor este returnată în variabila *array_of_statuses* și fiecare poate fi interogat în mod obișnuit pentru determinarea sursei sau *tag*-ului, dacă este necesar. Această funcție există și în forma de comunicație blocantă. Forma sa este:

int MPI_Testall (int *count, MPI_Request array_of_requests[], bool flag, MPI_Status array_of_statuses[]) . Dacă toate comunicațiile s-au terminat, variabila *flag* este setată ca *true* și informațiile despre fiecare comunicație sunt returnate în *array_of_statuses*, altfel *flag* este setat la *false* și *array_of_statuses* este nedefinit.

Când se testează un număr generic de comunicații, se utilizează funcția:

int MPI_Waitany (int count, MPI_Request array_of_requests, int index, MPI_Status status) . MPI_Waitany se blochează până când o comunicație asociată cu *array_of_requests* s-a încheiat. Indexul comunicației încheiate este returnat (variabila *index*), alături de starea sa (variabila *status*). Dacă mai multe comunicații se încheie în același timp, indexul returnat este arbitrar. Testarea fără blocare se realizează cu funcția: **int MPI_Testany (int count, MPI_Request array_of_requests, int index, bool flag, MPI_Status status)**.

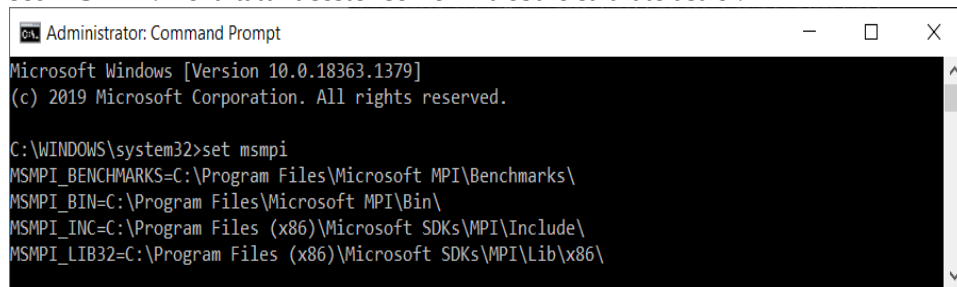
Funcția *MPI_Testsome*, spre deosebire de *MPI_Testany*, acceptă ca mai multe comunicații să se termine în același timp. Această funcție returnează un vector cu indici și altul cu stările comunicațiilor terminate.

Sintaxa acestei funcții este: **int MPI_Testsome (int incount, MPI_Request array_of_requests[], int outcount, int array_of_indices[], MPI_Status array_of_statuses[])**.

Instalarea și utilizarea bibliotecii MSMPI în .NET (limbajul VC++)

Pentru utilizarea bibliotecii MPI în .NET se parcurg următorii pași:

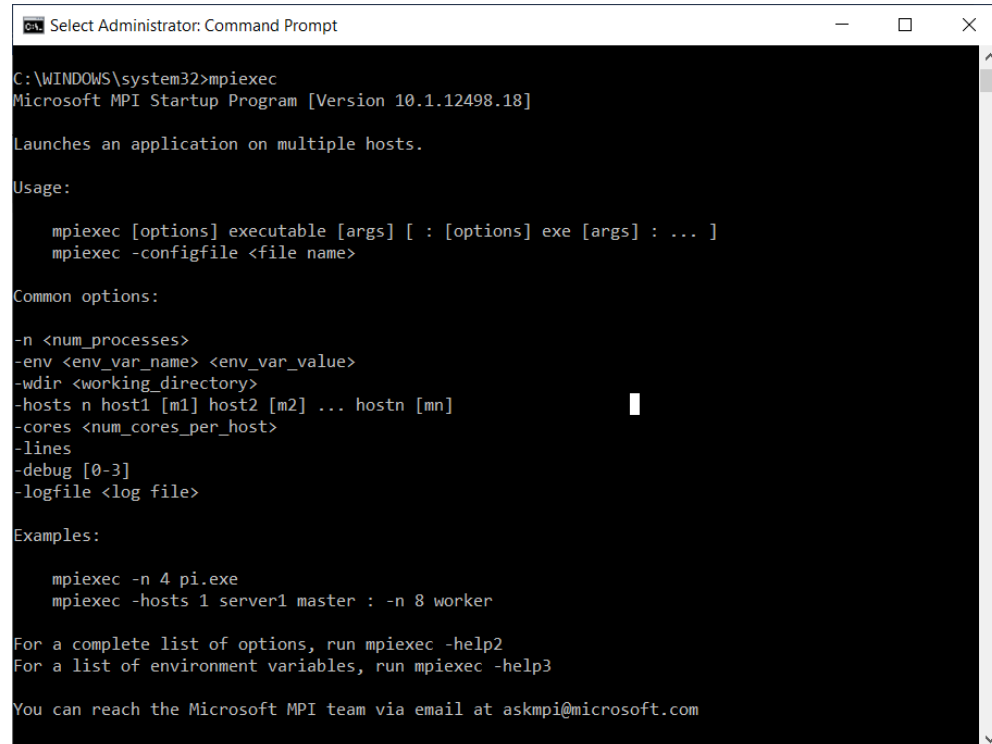
1. Se verifică dacă există alte versiuni deja instalate prin comanda cmd: **where mpiexec** și **where smpd**. Dacă se primește un rezultat în urma apelării acestor comenzi, librăriile existente trebuie dezinstate.
2. Se instalează **msmpisetup.exe**.
3. Se instalează **msmpisdk.exe**.
4. Se verifică dacă variabilele de sistem au fost stabilite corect prin comanda cmd: **set MSMPI**. Rezultatul acestei comenzi trebuie să arate astfel:



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.18363.1379]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>set mspi
MSMPI_BENCHMARKS=C:\Program Files\Microsoft MPI\Benchmarks\
MSMPI_BIN=C:\Program Files\Microsoft MPI\Bin\
MSMPI_INC=C:\Program Files (x86)\Microsoft SDKs\MPI\Include\
MSMPI_LIB32=C:\Program Files (x86)\Microsoft SDKs\MPI\Lib\x86\
```

5. Se verifică instalarea corectă a librăriei MPI prin comanda cmd: **mpiexec**. Rezultatul acestei comenzi trebuie să arate astfel:



```
Select Administrator: Command Prompt
C:\WINDOWS\system32>mpiexec
Microsoft MPI Startup Program [Version 10.1.12498.18]

Launches an application on multiple hosts.

Usage:

    mpiexec [options] executable [args] [ : [options] exe [args] : ... ]
    mpiexec -configfile <file name>

Common options:

-n <num_processes>
-env <env_var_name> <env_var_value>
-wdir <working_directory>
-hosts n host1 [m1] host2 [m2] ... hostn [mn]
-cores <num_cores_per_host>
-lines
-debug [0-3]
-logfile <log file>

Examples:

    mpiexec -n 4 pi.exe
    mpiexec -hosts 1 server1 master : -n 8 worker

For a complete list of options, run mpiexec -help2
For a list of environment variables, run mpiexec -help3

You can reach the Microsoft MPI team via email at askmpi@microsoft.com
```

6. Pentru includerea MPI în .NET (limbajul VC++) se parcurg următoarele etape:

- Se deschide **Microsoft Visual Studio**. Crearea unui proiect nou se face selectând comanda **New Project** din fereastra principală (figura 1.17).

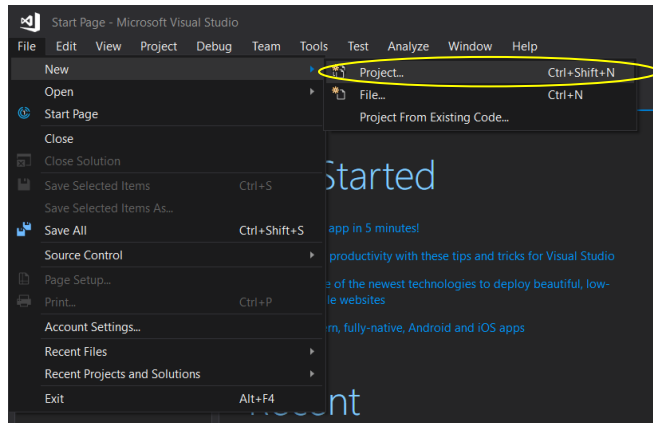


Figura 1.9 Mediul de dezvoltare .NET

- În figura 1.18 prezentăm caseta de dialog **NewProject**.

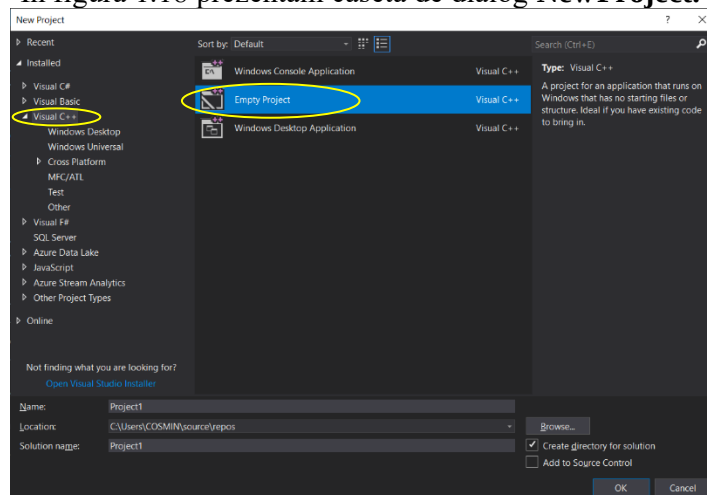


Figura 1.10 Caseta de dialog New Project

- Selectarea tipului de proiect se face alegând opțiunea **Visual C++** din arborele din stânga. În partea dreaptă a ferestrei va apărea o listă cu mai multe tipuri de proiecte. Pentru acest exemplu selectați **Empty Project** din lista cu tipurile de

proiecte posibile. Selectarea acestei opțiuni înseamnă că proiectul va avea ca rezultat un program executabil.

- **Denumirea proiectului și alegerea locației sale.** Orice proiect are nevoie de un nume. Acest nume se specifică în caseta **Name** din fereastra **New Project**. Pentru exemplul din acest laborator, în caseta **Name** tastați numele „Laborator1”.
- Se adaugă o sursă nouă la proiect. Se selectează opțiunea **Project > Add New Item**. Se selectează o sursă C++ File (.cpp).

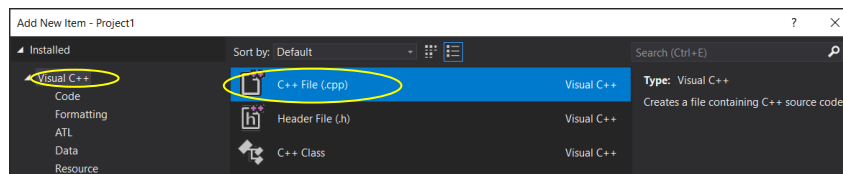


Figura 1.21 Adăugare sursă nouă în proiect

- Se selectează opțiunea **Project > Properties** pentru alegerea proprietăților pentru acest proiect. Se schimbă o parte din setările implicite ale proiectului pentru a se putea utiliza funcțiile din librăria msmapi.

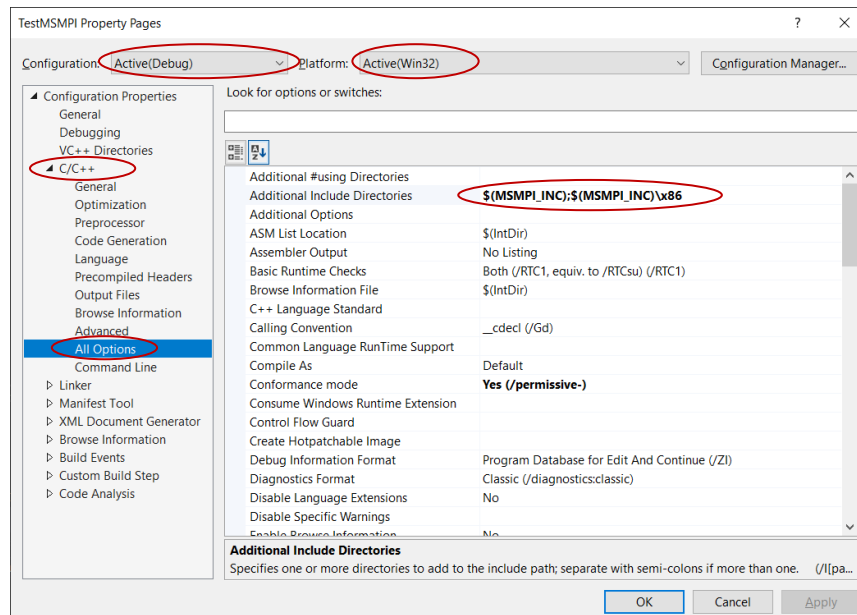


Figura 1.12 Setările pentru Debug

- Se includ căile către librăriile din mpi. Se adaugă și librăriile `ws2_32.lib` (Microsoft Winsock2 Library)

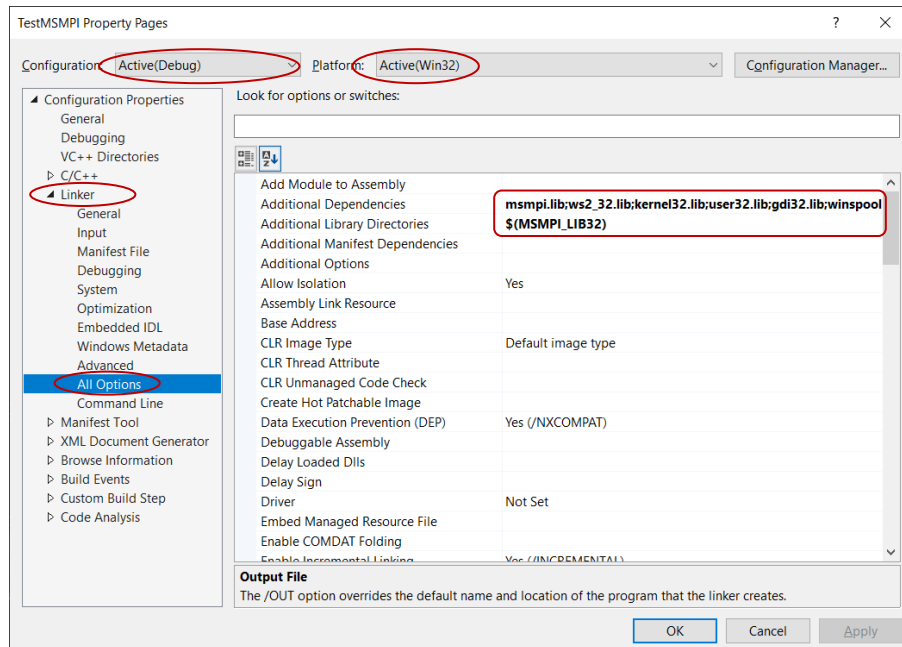


Figura 1.13 *Includerea căilor către librăriile MPI*

- Se închide caseta cu proprietățile proiectului.
- Se scrie codul în fișierul sursă al proiectului (.cpp).
- Se compilează pentru erori. Dacă nu sunt erori în urma compilării mediul Visual Studio va crea un fișier executabil cu numele proiectului.

Executarea programelor MPI

După crearea executabilului se verifică funcționarea programului scris. Pentru aceasta există 2 metode:

Există posibilitatea executării local sau în rețea (tip cluster).

- Sintaxa instrucțiune mod LOCAL:

`mpiexec -n <nr_procese> NumeFisierExecutabil.exe`

- Pentru executarea în rețea este nevoie mai întâi de pornirea serviciului de comunicație pentru MPI. Aceasta se face cu comanda **smpd -d 3**. Acesta este un serviciu de management al MPI creat de Intel care permite pornirea și gestionarea task-urilor paralele.

Sintaxa instrucțiune mod RETEA:

`mpiexec -hosts <nr_host-uri> numeHost_i nrProceseHost_i
NumeFisierExecutabil.exe`

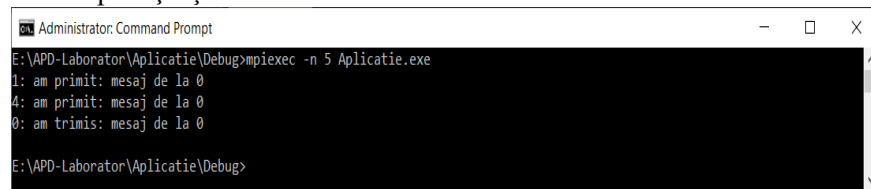
Exemplu:

`mpiexec -hosts 2 localhost 2 192.168.96.96 5 TestMSMPI.exe`

ATENȚIE: Pe toate host-urile (PC) proiectul (fișierul executabil) trebuie să fie la aceeași locație!

Varianta 1 – Command Prompt (cmd)

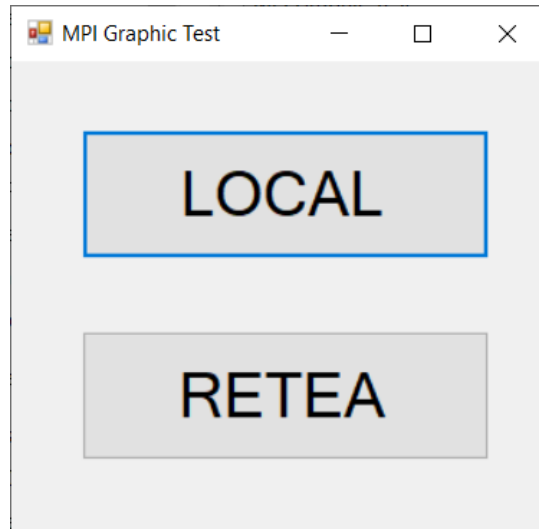
- Se deschide Command Prompt (cmd).
- Se scriu și se execută instrucțiunile **mpiexec ...** în funcție de aplicație și cerințele acesteia.



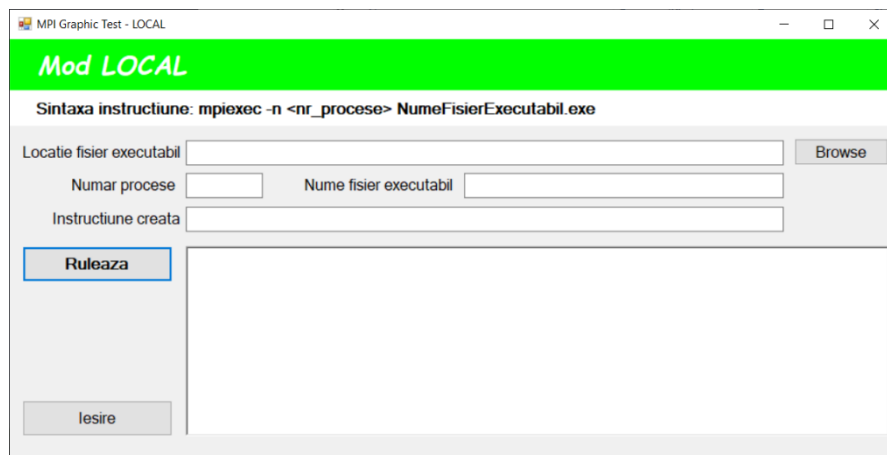
```
Administrator: Command Prompt
E:\APD-Laborator\Aplicatie\Debug>mpiexec -n 5 Aplicatie.exe
1: am primit: mesaj de la 0
4: am primit: mesaj de la 0
0: am trimis: mesaj de la 0
E:\APD-Laborator\Aplicatie\Debug>
```

Varianta 2 – MPI Graphic Test (interfață grafică)

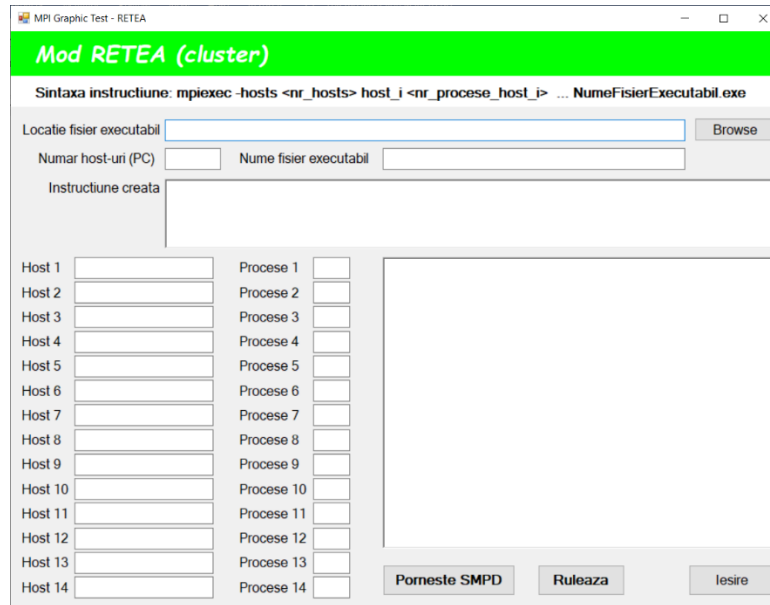
- Se deschide aplicația MPI Graphic Test.
- Se selectează modul de test: LOCAL sau RETEA.



- Varianta interfața în mod LOCAL.



- Varianta interfața în mod RETEA (cluster).



Aplicații în MPI

1. În cele ce urmează prezentăm un exemplu de program MPI ce implementează o operație de comunicație în inel. Procesorul P_0 va transmite un mesaj vecinilor săi P_1 și P_{p-1} . În acest exemplu, procesul 0 ($my_id == 0$) trimite un mesaj proceselor P_1 și P_{p-1} folosind operația *send*. Celelalte procese ($my_id == 1 \parallel my_id == p-1$) primesc acest mesaj prin operația *recv* și îl afișează. Descrierile celor trei procese coincid, iar diferențierea între acțiunile lor se face prin selecția după numărul de ordine al procesului. Eticheta asociată mesajului este 99, aceeași la sursă și destinație. Spre exemplu dacă $P=6$ (avem 6 procese), programul afișează mesajul din figura 1.25.

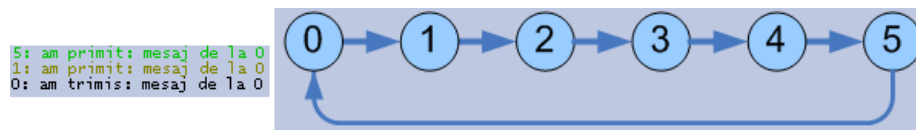


Figura 1.14 Rezultatul execuției programului paralel

Codul sursă al aplicației este următorul:

```
#include <iostream>
#include <mpi.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char**argv)
{
    int my_id; //identificatorul procesului
    int p;     //numărul de procese
    MPI_Status stare; //starea unui recepții
    char mesaj[100]; //mesajul de trimis

    MPI_Init(&argc, &argv); //initializare MPI
    MPI_Comm_rank(MPI_COMM_WORLD, &my_id); //adresa procese
    MPI_Comm_size(MPI_COMM_WORLD, &p); //nr. procese

    if (p < 3) //daca sunt mai puțin de 3 procese
    {
        if (my_id == 0) //doar procesul 0 afișează mesaj de eroare
            printf("Sunt necesare cel puțin 3 procesoare!\n");
        exit(0);
    }
    if (my_id == 0)
    {
        //construire mesaj
        sprintf_s(mesaj, "mesaj de la %d", my_id);
        //trimitere mesaj
        MPI_Send(mesaj, strlen(mesaj) + 1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
        MPI_Send(mesaj, strlen(mesaj) + 1, MPI_CHAR, p - 1, 99,
MPI_COMM_WORLD);
        // afisare mesaj
        printf("%d: am trimis: %s\n", my_id, mesaj);
    }
    else if (my_id == 1 || my_id == p - 1)
    {
        MPI_Recv(mesaj, 50, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &stare);
        printf("%d: am primit: %s\n", my_id, mesaj);
    }
    // Finalizarea procesului MPI
    MPI_Finalize();
    return 0;
}
```

Aplicația prezentată mai sus se compilează cu Microsoft Visual Studio și după obținerea executabilului se va rula sub mediul MPI.

2. Prezentăm un exemplu de program MPI ce implementează operația de transmitere a unui mesaj de către **fiecare** procesor vecinilor săi (se consideră o topologie de tip inel).

Codul sursă al aplicației este următorul:

```
#include <iostream>
#include <mpi.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char**argv)
{
    int my_id;
    int p;
    int dreapta, stanga;
    MPI_Status stare;
    char msend[100], mrecv[100], buffer[1000];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    dreapta = (my_id + 1) % p;
    stanga = (my_id - 1 + p) % p;

    sprintf(msend, "mesaj de la %d", my_id);
    MPI_Buffer_attach(buffer, 1000);
    MPI_Bsend(msend, strlen(msend) + 1, MPI_CHAR, dreapta, 99,
MPI_COMM_WORLD);
    MPI_Recv(mrecv, 100, MPI_CHAR, stanga, 99, MPI_COMM_WORLD, &stare);

    printf("%d: am primit: %s\n", my_id, mrecv);

    MPI_Finalize();
    return 0;
}
```

În această aplicație, pentru trimiterea mesajelor, am folosit funcția de comunicație blocantă prin buffer *MPI_Bsend*. Eticheta asociată mesajului este 99, aceeași la sursă și destinație.

Desfășurarea lucrării:

1. Se va citi breviarul teoretic. Se atrage atenția asupra faptului că toate cunoștințele din această lucrare vor fi necesare și în derularea celorlalte lucrări.
2. Se vor studia problemele rezolvate (problemele prezentate pe parcursul acestui laborator precum și cele oferite ca exemple în kitul MPI), încercând găsirea altor posibilități de soluționare a acestora.
3. Programele prezentate în laborator (problemele rezolvate) vor fi scrise în *Visual C++* și se vor rula utilizând directivele din librăria *MPI*. Se va observa efectul rulării și buna lor execuție.
4. Se vor rezolva următoarele probleme propuse și se va urmări execuția lor corectă.
 - Modificați programul 1 prezentat în lucrare (Aplicația 1) astfel încât procesorul P_0 să transmită un mesaj tuturor procesoarelor conectate în rețeaua respectivă (procesoarele sunt conectate într-o structură de tip inel).
 - Modificați programul 2 prezentat în lucrare (Aplicația 2) înlocuind comunicația blocantă prin buffer (*MPI_Bsend*) cu: *MPI_Ssend*, *MPI_Isend*, *MPI_Issend*, *MPI_Ibsend*.